



رقم

29

4350 sp

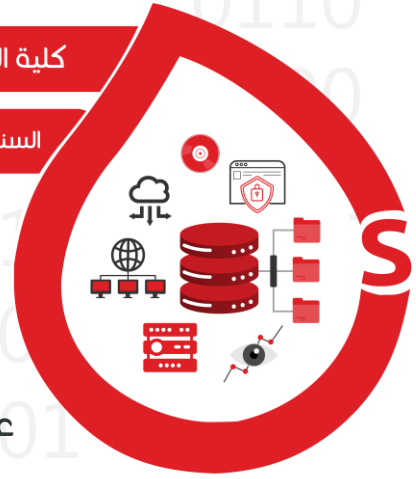
ملحق

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

ملحق عملي

عمل طلابي



RB Informatics; 14/1/2026

محتوى مجاني غير مخصص للبيع التجاري

قواعد معطيات ٢

نضع بين أيديكم ملحق لمادة قواعد المعطيات ٢ وهو عمل طلابي بحث ولا يمكن الاستغناء عن دراسة المحاضرات، نرجو لكم التوفيق.

Part 1: Syntax Cheat Sheet

أوامر DML – Data Manipulation Language

الأمور	الوصف	الصيغة (Syntax)	مثال
SELECT	جلب البيانات من الجدول	SELECT col1, col2 FROM table;	SELECT name, age FROM students;
INSERT	إدخال سجل جديد	INSERT INTO table(col1,col2) VALUES (v1,v2);	INSERT INTO students(name,age) VALUES('Ali',20);
UPDATE	تعديل بيانات موجودة	UPDATE table SET col=value WHERE condition;	UPDATE students SET age=21 WHERE id=1;
DELETE	حذف سجلات	DELETE FROM table WHERE condition;	DELETE FROM students WHERE age < 18;

أوامر DDL – Data Definition Language

الأمور	الوصف	الصيغة (Syntax)	مثال
CREATE TABLE	إنشاء جدول	CREATE TABLE t (col datatype);	CREATE TABLE users(id INT, name VARCHAR (100));
ALTER TABLE	تعديل بنية جدول	ALTER TABLE t ADD col datatype;	ALTER TABLE users ADD email VARCHAR(100);
DROP TABLE	حذف جدول بالكامل	DROP TABLE t;	DROP TABLE users;
TRUNCATE	حذف البيانات فقط	TRUNCATE TABLE t;	TRUNCATE TABLE logs;

أوامر الاستعلام (WHERE, GROUP BY, HAVING, ORDER BY)

العنصر	الوصف	مثال
WHERE	تصفية الصفوف	SELECT * FROM emp WHERE salary > 500;
ORDER BY	ترتيب النتائج	SELECT * FROM emp ORDER BY salary DESC;
GROUP BY	تجميع البيانات	SELECT dept, COUNT(*) FROM emp GROUP BY dept;
HAVING	شرط على التجميع	SELECT dept, COUNT() FROM emp GROUP BY dept HAVING COUNT () > 5;

JOINS

العنصر	الوصف	مثال
INNER JOIN	صفوف مشتركة فقط	emp INNER JOIN dept ON emp.did=dept.id
LEFT JOIN	كل اليسار + المشترك	emp LEFT JOIN dept ON emp.did=dept.id
RIGHT JOIN	كل اليمين + المشترك	emp RIGHT JOIN dept ON emp.did=dept.id
CROSS JOIN	حاصل ضرب ديكارتي	emp CROSS JOIN dept
SELF JOIN	ربط الجدول بنفسه	emp e1 JOIN emp e2

الدوال التجميعية (Aggregate Functions)

العنصر	الوصف	مثال
COUT	عدد الصفوف	SELECT COUNT(*) FROM orders;
SUM	مجموع القيم	SELECT SUM(price) FROM sales;
AVG	المتوسط	SELECT AVG(score) FROM exams;
MIN	أصغر قيمة	SELECT MIN(age) FROM users;
MAX	أكبر قيمة	SELECT MAX(age) FROM users;

الدوال (Functions)

العنصر	الشرح
Function	كود يعيد قيمة واحدة أو جدول واحد ويمكن استخدامه داخل SELECT

1. Scalar Function (دالة تعيد قيمة واحدة)

- تعيد قيمة واحدة فقط (رقم، نص، تاريخ).
- يمكن استخدامها داخل SELECT أو WHERE.

Syntax:

```
CREATE FUNCTION function_name
(
  @parameter_name parameter_data_type [ = default ] → يمكن إعطاء قيمة أولية/افتراضية
  للبارميتر
)
RETURNS return_data_type
AS
BEGIN
  function_body
RETURN scalar_expression
END;
```

Example:

```
CREATE FUNCTION dbo.get_bonus(@salary INT)
RETURNS INT
AS
BEGIN
  RETURN @salary * 0.1;
END;
SELECT name, dbo.get_bonus(salary) AS bonus
FROM employees;
```

2. Inline Table-Valued Function (دالة جدول – سطر واحد)

▪ تعيد جدول كامل وتحتوي على SELECT واحد فقط

Syntax:

```
CREATE FUNCTION function_name
(
  @parameter_name parameter_data_type
)
RETURNS TABLE
AS
RETURN (
  Select statement
);
```

Example:

```
CREATE FUNCTION dbo.get_active_users()
RETURNS TABLE
AS
RETURN (
SELECT id, name
FROM users
WHERE status = 'active'
);
```

العروض (View)

العنصر	الشرح
View	جدول افتراضي مبني على استعلام

Syntax:

```
CREATE VIEW view_name AS
single SELECT statement;
```

ملاحظة:

ال View لا يخزن بيانات، وإنما يخزن الاستعلام فقط، ويتم تنفيذ هذا الاستعلام من جديد عند كل طلب، لذلك تكون البيانات دائماً محدثة.

Example:

```
CREATE VIEW active_users AS
SELECT * FROM users WHERE status = 'active';
```

الإجراءات المخزنة (Stored Procedures)

العنصر	الشرح
Procedure	مجموعة أوامر SQL تُنفَّذ عند الطلب

Syntax:

```
CREATE PROCEDURE procedure_name
(
@parameter_name parameter_data_type
)
AS
BEGIN
sql_statement
END
```

And to execute it (ولتنفيذها):

EXEC procedure_name;

Example:

```
CREATE PROCEDURE add_user
@name VARCHAR(50)
AS
INSERT INTO users(name) VALUES(@name);

--execute the procedure
EXEC add_user 'Ahmed';
```

المعاملات (Transaction)

الشرح	العنصر
بدء معاملة	BEGIN TRANSACTION
تثبيت التغييرات	COMMIT
التراجع عن التغييرات	ROLLBACK

Example:

```
BEGIN TRANSACTION;

INSERT INTO orders VALUES(1,100);
UPDATE products SET qty = qty-1 WHERE id=5;

COMMIT;
```

Triggers

الشرح	العنصر
كود SQL يُنفَّذ تلقائياً عند حدوث حدث معين على جدول أو View مثل INSERT أو UPDATE أو DELETE	Trigger

Syntax:

```
CREATE TRIGGER trigger_name
ON table_name
AFTER | INSTEAD OF
INSERT | UPDATE | DELETE
AS
BEGIN
    sql_statement
END;
```

ملاحظات:

- AFTER: يُنفَّذ بعد تنفيذ التعليمة.
- INSTEAD OF: يُنفَّذ بدل التعليمة.
- يتم تشغيله تلقائياً عند حدوث الحدث المرتبط به (المقصود بالحدث هو عملية الإضافة أو الحذف أو التعديل)
- Trigger لا يأخذ Parameters

Example:

```
CREATE TRIGGER trg_after_insert_employee
ON employees
AFTER INSERT
AS
BEGIN
    INSERT INTO audit_log(action, action_date)
    VALUES ('New employee inserted', GETDATE());
END;
```

هذا ال Trigger يعمل تلقائياً عند تنفيذ:

```
INSERT INTO employees(name, salary)
VALUES ('Ahmed', 1200);
```

فائدة اربيسيزية

Auditing (التدقيق وتتبع التغييرات):

في كثير من الأنظمة، لا يكفي تعديل البيانات فقط، بل نحتاج إلى تتبع ما الذي تغير، ومتى تغير، ومن قام بالتغيير. هذه العملية تسمى Auditing

لماذا نستخدم Auditing؟

- تتبع التعديلات على البيانات الحساسة (مثل الرواتب).
 - معرفة القيم القديمة والجديدة.
 - أغراض أمنية أو إدارية.
 - تصحيح الأخطاء أو التحقيق لاحقاً.
- في SQL Server، تُعد Triggers من أكثر الطرق شيوعاً لتنفيذ ال Auditing.

نهاية الفائدة الاريسيزية

دور deleted و inserted في Triggers:

عند تنفيذ Trigger في SQL Server، يوفر النظام جدولين منطقيين مؤقتين:

1. inserted

- يحتوي على القيم الجديدة
- يُستخدم مع: INSERT و UPDATE

2. deleted

- يحتوي على القيم القديمة
- يُستخدم مع: DELETE و UPDATE

في حالة UPDATE:

- deleted يحوي القيم قبل التعديل.
- inserted يحوي القيم بعد التعديل.

في INSERT:

- فقط inserted يحتوي بيانات.

في DELETE:

- فقط deleted يحتوي بيانات.

ملاحظة مهمة:

deleted و inserted ليسا جداول حقيقية، بل جداول منطقية مؤقتة متاحة فقط داخل Trigger.

Example:

Auditing باستخدام deleted و inserted

لننشئ جدول التتبع (Audit Table):

```
CREATE TABLE salary_audit (
    emp_id INT,
    old_salary INT,
    new_salary INT,
    change_date DATETIME
);
```

:Trigger (AFTER UPDATE)

```
CREATE TRIGGER trg_update_salary
ON employees
AFTER UPDATE
AS
BEGIN
    INSERT INTO salary_audit(emp_id, old_salary, new_salary, change_date)
    SELECT
        i.emp_id,
        d.salary AS old_salary,
        i.salary AS new_salary,
        GETDATE()
    FROM inserted i
    JOIN deleted d
    ON i.emp_id = d.emp_id;
END;
```

تنفيذ المثال:

```
UPDATE employees
SET salary = 1500
WHERE emp_id = 3;
```

عند تنفيذ هذا الأمر:

- يعمل ال Trigger تلقائياً.
- يتم حفظ الراتب القديم والجديد في جدول salary_audit

معالجة الأخطاء (TRY / CATCH)

العنصر	الشرح
TRY / CATCH	آلية لمعالجة الأخطاء في SQL Server ومنع فشل التنفيذ بالكامل عند حدوث خطأ

تستخدم مع Transactions لضمان:

- إما تنفيذ جميع الأوامر بنجاح
- أو التراجع عنها بالكامل عند حدوث خطأ

Syntax:

```

BEGIN TRY
    sql_statements
END TRY
BEGIN CATCH
    *ملاحظة: مع ال transaction نضع ال ROLLBACK هنا
    error_handling_statements
END CATCH;
```

ملاحظات مهمة:

- لا تستخدم TRY / CATCH داخل VIEW أو FUNCTION
- هي الطريقة الصحيحة لاستخدام Transactions في SQL Server.

مثال مهم TRY / CATCH مع Transactions:

```

BEGIN TRY
    BEGIN TRANSACTION;

    INSERT INTO orders(order_id, total)
    VALUES (1, 500);

    UPDATE products
    SET quantity = quantity - 1
    WHERE product_id = 10;

    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
    PRINT 'Transaction failed and rolled back';
END CATCH;
```

الحلقة التكرارية (WHILE LOOP)

العنصر	الشرح
WHILE	حلقة تكرارية تُستخدم لتنفيذ أوامر SQL عدة مرات طالما الشرط محقق

Syntax:

```
WHILE condition
BEGIN
    sql_statements
END;
```

مثال مهم:

إضافة عدد كبير من البيانات بتعليم إدخال واحدة

السيناريو:

إضافة عدد كبير من الإشعارات (Notifications) أو البيانات التجريبية إلى جدول واحد.

```
DECLARE @counter INT = 1;

WHILE @counter <= 1000
BEGIN
    INSERT INTO notifications(message, created_at)
    VALUES ('System maintenance notification', GETDATE());

    SET @counter = @counter + 1;
END;
```

الفهارس (Indexes)

العنصر	الشرح
Index	بنية بيانات تُستخدم لتحسين سرعة الوصول إلى البيانات وتقليل زمن تنفيذ الاستعلامات

أنواع الفهارس في SQL Server:

1. Clustered Index:

- يحدد الترتيب الفيزيائي للبيانات داخل الجدول
- لا يمكن أن يوجد أكثر من Clustered Index واحد لكل جدول
- غالبًا يُنشأ تلقائيًا على Primary Key

Syntax + Example:

```
CREATE CLUSTERED INDEX idx_emp_id
ON employees(emp_id)
```

2. Non-Clustered Index:

- لا يغيّر ترتيب البيانات داخل الجدول.
- يحتوي على:
 - مفتاح الفهرس
 - Pointer إلى الصف الفعلي
- يمكن إنشاء عدة Non-Clustered Indexes على نفس الجدول:

Syntax + Example:

```
CREATE NONCLUSTERED INDEX idx_emp_name  
ON employees(name);
```

3. Composite Index (فهرس مركّب):

- فهرس مبني على أكثر من عمود
- ترتيب الأعمدة مهم جداً

Example:

```
CREATE NONCLUSTERED INDEX idx_emp_dept_salary  
ON employees(department_id, salary);
```

يُفيد عندما يكون الاستعلام يستخدم الأعمدة بنفس الترتيب.

4. Unique Index:

- يمنع تكرار القيم.
- يمكن أن يكون Clustered أو Non-Clustered

Example:

```
CREATE UNIQUE INDEX idx_user_email  
ON users(email);
```

5. Filtered Index:

- فهرس على جزء من البيانات فقط.
- مفيد جداً لتحسين الأداء وتقليل الحجم

Example:

```
CREATE NONCLUSTERED INDEX idx_active_users  
ON users(status)  
WHERE status = 'active';
```

6. Covering Index (الفهرس المغطّي):

معلومة مهمة جداً:

Covering Index ليس نوع فهرس، بل صفة (Property) يمكن أن تطلق على الفهرس:

- هو فهرس يحتوي كل الأعمدة المطلوبة في الاستعلام.
- يمنع الحاجة للرجوع إلى الجدول (No Key Lookup)

Syntax (include باستخدام):

```
CREATE NONCLUSTERED INDEX idx_order_cover
ON orders(order_id)
INCLUDE (order_date, total_amount);
```

إذا كان الاستعلام يستخدم فقط الأعمدة الموجودة في الفهرس فهو يعتبر Covering.

فائدة ارييسيزية

ما هي T-SQL ؟

T-SQL (Transact-SQL) هي امتداد خاص من لغة SQL تستخدمه **SQL Server**، وتضيف إمكانيات برمجية مثل:

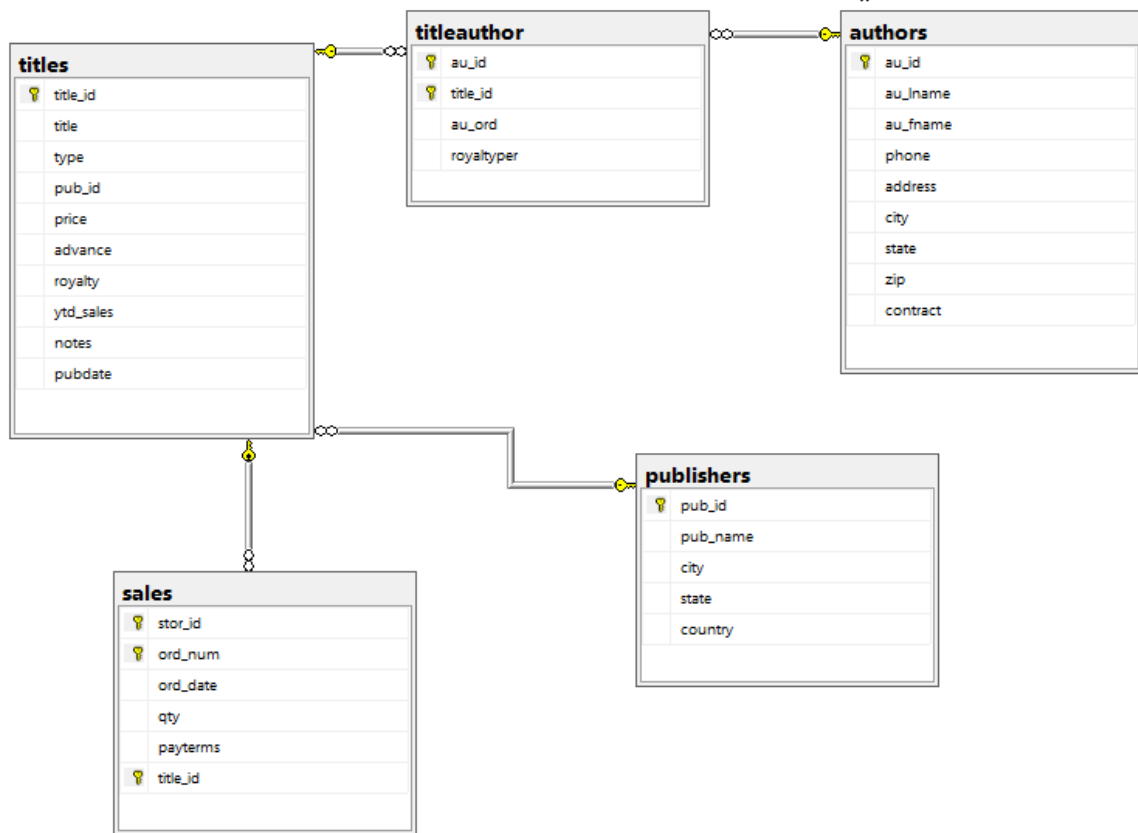
- المتغيرات
- الشروط (IF)
- الحلقات (WHILE)
- الإجراءات المخزنة (Stored Procedures)
- معالجة الأخطاء (TRY / CATCH)
- المعاملات (Transactions)

تستخدم T-SQL لكتابة منطق برمجي داخل قاعدة البيانات، مما يساعد على **تنفيذ عمليات معقدة بكفاءة أعلى وأمان أكبر.**

نهاية الفائدة الاريسيزية

تدريب على الجزء الأول

ليكن لدينا قاعدة البيانات المبينة في المخطط:



1. بلغة T-SQL قم بكتابة إجرائية (Stored Procedure) تقوم بعرض:

- اسم الكتاب
- رقم الكتاب
- الاسم الكامل للمؤلف
- وذلك حسب نوع الكتاب الذي يُمرَّر للإجرائية.

الحل:

```

CREATE PROCEDURE sp_GetBooksByType
    @BookType VARCHAR(30)
AS
BEGIN
    SELECT
        t.title_id      AS [رقم الكتاب],
        t.title         AS [اسم الكتاب],
        a.au_fname + ' ' + a.au_lname AS [اسم المؤلف الكامل]
    FROM titles t
    INNER JOIN titleauthor ta ON t.title_id = ta.title_id
    INNER JOIN authors a ON ta.au_id = a.au_id
    WHERE t.type = @BookType;
END;
  
```

```
EXEC sp_GetBooksByType 'business';
```

5. Procedure تعرض جميع الكتب لمؤلف معين:

```
CREATE PROCEDURE sp_GetBooksByAuthor
    @AuthorID CHAR(11)
AS
BEGIN
    SELECT
        t.title_id,
        t.title
    FROM titles t
    INNER JOIN titleauthor ta
        ON t.title_id = ta.title_id
    WHERE ta.au_id = @AuthorID;
END;
```

6. عرض أنواع الكتب وعدد الكتب في كل نوع (أكثر من 2 فقط):

```
SELECT
    type,
    COUNT(*) AS books_count
FROM titles
GROUP BY type
HAVING COUNT(*) > 2;
```

7. عرض مبيعات كل كتاب (إجمالي الكمية):

```
SELECT
    t.title,
    SUM(s.qty) AS total_sold
FROM titles t
INNER JOIN sales s
    ON t.title_id = s.title_id
GROUP BY t.title;
```

2. عرض أسماء الكتب مع اسم دار النشر:

```
SELECT
    t.title,
    p.pub_name
FROM titles t
INNER JOIN publishers p
    ON t.pub_id = p.pub_id;
```

3. عرض أسماء المؤلفين وعدد الكتب لكل مؤلف:

```
SELECT
    a.au_fname + ' ' + a.au_lname AS author_name,
    COUNT(ta.title_id) AS books_count
FROM authors a
    INNER JOIN titleauthor ta
        ON a.au_id = ta.au_id
GROUP BY a.au_fname, a.au_lname;
```

4. View تعرض الكتب التي سعرها أكبر من 20:

```
CREATE VIEW vw_ExpensiveBooks AS
SELECT title_id, title, price
FROM titles
WHERE price > 20;
```

8. Function تعيد إجمالي مبيعات كتاب معيّن:

```

CREATE FUNCTION fn_TotalSales(@TitleID CHAR(6))
RETURNS INT
AS
BEGIN
    DECLARE @Total INT;

    SELECT @Total = SUM(qty)
    FROM sales
    WHERE title_id = @TitleID;

    RETURN ISNULL(@Total, 0);
END;

```

ملاحظة: تم استخدام ISNULL هنا لأنّه في حال عدم وجود أي سجلات مطابقة يكون ناتج SUM هو NULL، واستخدام RETURN ISNULL(@Total, 0) يضمن إرجاع القيمة 0 بدل NULL لتفادي نتائج غير متوقعة أو أخطاء منطقية عند استخدام الدالة.

9. Procedure لإضافة كتاب جديد باستخدام Transaction:

```

CREATE PROCEDURE sp_AddTitle
    @title_id CHAR(6),
    @title VARCHAR(80),
    @type VARCHAR(12),
    @pub_id CHAR(4),
    @price DECIMAL(10,2)
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;

        INSERT INTO titles(title_id, title, type, pub_id, price)
        VALUES (@title_id, @title, @type, @pub_id, @price);

        COMMIT;
    END TRY
    BEGIN CATCH
        ROLLBACK;
    END CATCH;
END;

```

10. Trigger لتسجيل تغيير السعر (Auditing):

```
CREATE TABLE price_audit (  
    title_id CHAR(6),  
    old_price DECIMAL(10,2),  
    new_price DECIMAL(10,2),  
    change_date DATETIME DEFAULT GETDATE()  
);
```

```
CREATE TRIGGER trg_TitlePriceUpdate  
ON titles  
AFTER UPDATE  
AS  
BEGIN  
    INSERT INTO price_audit(title_id, old_price, new_price)  
    SELECT  
        d.title_id,  
        d.price,  
        i.price  
    FROM deleted d  
    INNER JOIN inserted i  
        ON d.title_id = i.title_id  
    WHERE d.price <> i.price;  
END;
```

11. WHILE Loop لإدخال بيانات تجريبية:

```
DECLARE @i INT = 1;  
  
WHILE @i <= 5  
BEGIN  
    INSERT INTO titles(title_id, title)  
    VALUES ('T00' + CAST(@i AS CHAR), 'Test Book ' + CAST(@i AS VARCHAR));  
  
    SET @i = @i + 1;  
END;
```

12. اكتب Stored Procedure تستقبل:

- رقم دار النشر pub_id
 - وتعرض
 - اسم الكتاب
 - سعره
- لكل الكتب التابعة لهذه الدار.

```
CREATE PROCEDURE sp_GetBooksByPublisher
    @PubID CHAR(4)
AS
BEGIN
    SELECT
        t.title AS [اسم الكتاب],
        t.price AS [سعر الكتاب]
    FROM titles t
    WHERE t.pub_id = @PubID;
END;
```

ولتنفيذها:

```
EXEC sp_GetBooksByPublisher '0736';
```

13. اكتب Stored Procedure تستقبل:

- رقم مؤلف au_id
- وتعرض
- اسم المؤلف الكامل
- عدد الكتب التي ألفها.

```
CREATE PROCEDURE sp_GetAuthorBooksCount
    @AuthorID CHAR(11)
AS
BEGIN
    SELECT
        a.au_fname + ' ' + a.au_lname AS [اسم المؤلف الكامل],
        COUNT(ta.title_id) AS [عدد الكتب]
    FROM authors a
    INNER JOIN titleauthor ta
    ON a.au_id = ta.au_id
    WHERE a.au_id = @AuthorID
    GROUP BY a.au_fname, a.au_lname;
END;
```

ولتنفيذها:

```
EXEC sp_GetAuthorBooksCount '172-32-1176';
```

14. اكتب Stored Procedure تستقبل:

- نوع كتاب type
- وتعرض:
- نوع الكتاب
- متوسط سعر الكتب من هذا النوع.

```
CREATE PROCEDURE sp_GetAveragePriceByType
    @BookType VARCHAR(12)
AS
BEGIN
    SELECT
        type AS [نوع الكتاب],
        AVG(price) AS [متوسط السعر]
    FROM titles
    WHERE type = @BookType
    GROUP BY type;
END;
```

ولتنفيذها:

```
EXEC sp_GetAveragePriceByType 'business';
```

15. اكتب Stored Procedure تستقبل:

- رقم كتاب title_id
- وتعرض:
- اسم الكتاب
- إجمالي عدد النسخ المباعة له

```
CREATE PROCEDURE sp_GetTotalSalesByTitle
    @TitleID CHAR(6)
AS
BEGIN
    SELECT
        t.title AS [اسم الكتاب],
        SUM(s.qty) AS [إجمالي النسخ المباعة]
    FROM titles t
    INNER JOIN sales s
        ON t.title_id = s.title_id
    WHERE t.title_id = @TitleID
    GROUP BY t.title;
END;
```

ولتنفيذها:

```
EXEC sp_GetTotalSalesByTitle 'BU1032';
```

16. اكتب Stored Procedure تستقبل:

- تاريخاً @StartDate
- تاريخاً @EndDate
- وتعرض:
- أسماء الكتب التي تم بيعها ضمن هذا المجال الزمني.

```
CREATE PROCEDURE sp_GetBooksSoldBetweenDates
    @StartDate DATE,
    @EndDate DATE
AS
BEGIN
    SELECT DISTINCT
        t.title AS [اسم الكتاب]
    FROM titles t
    INNER JOIN sales s
        ON t.title_id = s.title_id
    WHERE s.ord_date BETWEEN @StartDate AND @EndDate;
END;
```

ولتنفيذها:

```
EXEC sp_GetBooksSoldBetweenDates '2023-01-01', '2023-12-31';
```

17. اكتب دالة تستقبل:

- رقم كتاب title_id
- وتُعيد:
- إجمالي عدد النسخ المباعة لهذا الكتاب من جدول sales.

```
CREATE FUNCTION fn_TotalSalesByTitle
(
    @TitleID CHAR(6)
)
RETURNS INT
AS
BEGIN
    DECLARE @Total INT;

    SELECT @Total = SUM(qty)
    FROM sales
    WHERE title_id = @TitleID;

    RETURN ISNULL(@Total, 0);
END;

SELECT dbo.fn_TotalSalesByTitle('BU1032') AS Total_Sold;
```

لاحظ استخدام الدالة داخل SELECT
(الدالة هنا تعيد قيمة)

18. اكتب دالة تستقبل:

- نوع كتاب type
- وتُعيد:
- متوسط سعر الكتب من هذا النوع

```
CREATE FUNCTION fn_AvgPriceByType
(
    @BookType VARCHAR(12)
)
RETURNS DECIMAL(10,2)
AS
BEGIN
    DECLARE @AvgPrice DECIMAL(10,2);

    SELECT @AvgPrice = AVG(price)
    FROM titles
    WHERE type = @BookType;

    RETURN @AvgPrice;
END;

SELECT dbo.fn_AvgPriceByType('business') AS Average_Price;
```

19. اكتب دالة تستقبل:

- رقم دار نشر pub_id
- وتُعيد جدولاً يحتوي على:
- رقم الكتاب
- اسم الكتاب
- سعره

```
CREATE FUNCTION fn_BooksByPublisher
(
    @PubID CHAR(4)
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        title_id,
        title,
        price
    FROM titles
    WHERE pub_id = @PubID
);

SELECT *
FROM dbo.fn_BooksByPublisher('0736');
```

لاحظ الدالة هنا تعيد جدول.

تُستخدم مثل جدول عادي داخل FROM.

20. اكتب دالة تستقبل:

- رقم مؤلف au_id
- وتُعيد جدولاً يحتوي على:
- رقم الكتاب
- اسم الكتاب الذي شارك هذا المؤلف في تأليفه.

```
CREATE FUNCTION fn_BooksByAuthor
(
    @AuthorID CHAR(11)
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        t.title_id,
        t.title
    FROM titles t
    INNER JOIN titleauthor ta
        ON t.title_id = ta.title_id
    WHERE ta.au_id = @AuthorID
);
SELECT *
FROM dbo.fn_BooksByAuthor('172-32-1176');
```

-لاحظ الاستخدام في SELECT...

21. اكتب دالة تستقبل:

- رقم مؤلف au_id
- وتُعيد:
- عدد الكتب المختلفة التي ألفها هذا المؤلف.

```
CREATE FUNCTION fn_CountBooksByAuthor
(
    @AuthorID CHAR(11)
)
RETURNS INT
AS
BEGIN
    DECLARE @Count INT;

    SELECT @Count = COUNT(DISTINCT title_id)
    FROM titleauthor
    WHERE au_id = @AuthorID;

    RETURN ISNULL(@Count, 0);
END;

SELECT dbo.fn_CountBooksByAuthor('172-32-1176') AS Books_Count;
```

Part2: execution plan

1. ماهي خطة التنفيذ (Execution Plan)؟

خطة التنفيذ هي: الخطة التي يضعها SQL Server لتحديد كيف سينفذ الاستعلام الذي كتبه المستخدم.

- المستخدم يكتب: ماذا يريد (SELECT)
 - SQL Server يقرر كيف يصل للنتيجة بأفضل كلفة.
- خطة التنفيذ توضح:

- ترتيب العمليات
- نوع كل عملية
- تكلفة كل خطوة
- كيف يتم الوصول للبيانات (Index / Scan / Join)

2. لماذا خطة التنفيذ مهمة؟

لأن:

- نفس الاستعلام يمكن تنفيذه بطرق مختلفة
- بعض الطرق بطيئة جداً
- وبعضها سريع جداً
- الفرق غالباً يكون بسبب:
 - Index
 - طريقة JOIN
 - وجود SORT أو Lookup

3. العمليات في خطة التنفيذ execution plan operators:

أهم العمليات التي يجب معرفتها:

Operator	متى تظهر؟	الفكرة المبسطة
Table Scan	عند عدم وجود فهرس مناسب أو عند قراءة معظم الجدول	قراءة الجدول كاملاً صفّاً صفّاً
Index Scan	عند وجود فهرس لكن بدون شرط انتقائي أو مع Range كبير	المرور على الفهرس بالكامل أو جزء كبير منه
Index Seek	عند وجود شرط مساواة أو نطاق محدود على أعمدة الفهرس	القفز مباشرة إلى الصفوف المطلوبة
RID Lookup	عند استخدام Heap (جدول بلا Clustered Index)	جلب صفوف من الجدول باستخدام RID

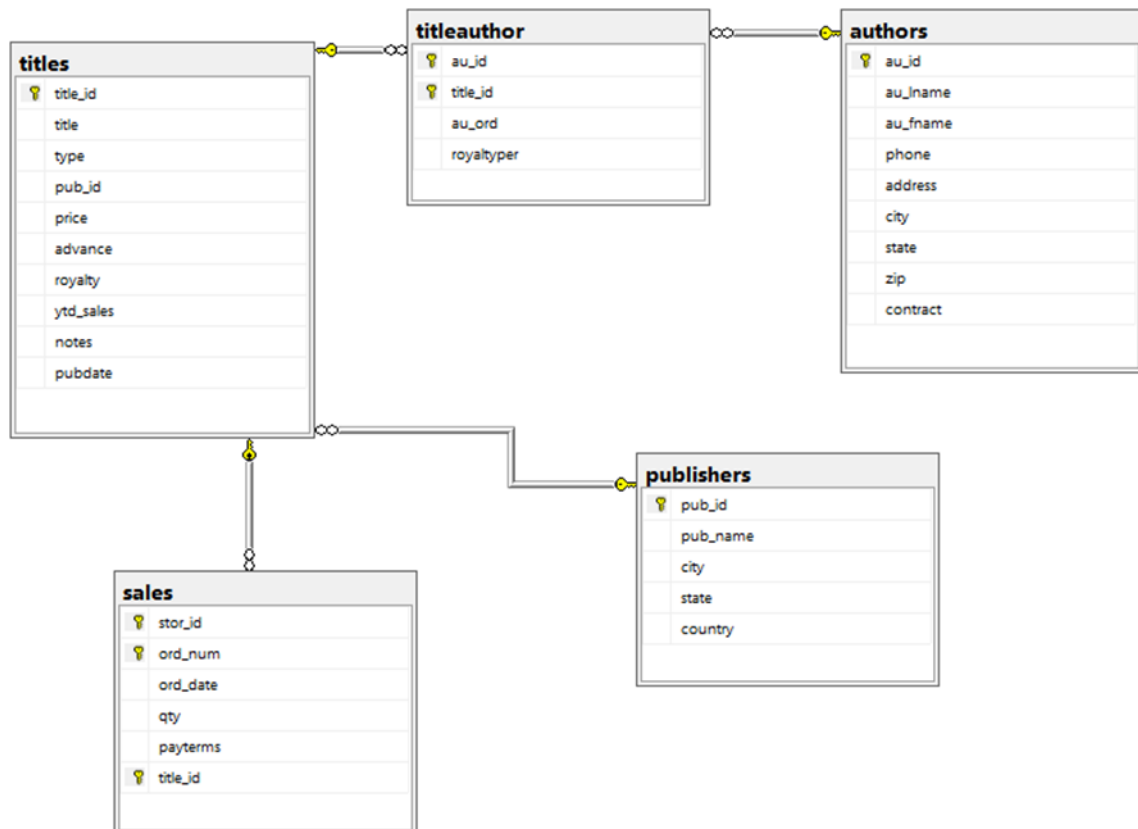
Key Lookup	عند استخدام Nonclustered Index غير مغطي	جلب أعمدة ناقصة من الجدول الأساسي
Hash Join	عند جداول كبيرة أو بدون ترتيب وفهارس مناسبة	مطابقة الصفوف باستخدام Hash
Merge Join	عند وجود ترتيب مسبق في الجدولين	الدمج السريع لبيانات مرتبة
Nested Loops	عند جدول صغير مع جدول آخر مفهرس	حلقة داخل حلقة
Sort	عند الحاجة لترتيب غير متوفر في الفهرس	ترتيب النتائج مؤقتًا

مثال:

ليكن لدينا قاعدة البيانات المستخدمة (pubs):
يوجد فيها الجداول الأساسية:

titles
authors

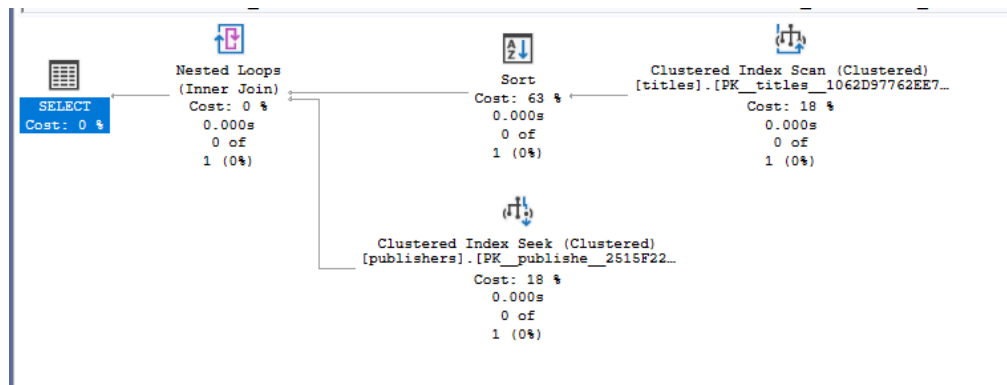
titleauthor
sales
publishers



بشكل تلقائي تم بناء فهارس من نوع clustered فقط على الجداول، قمنا بالاستعلام التالي وكانت خطة التنفيذ كما يلي:

```
SELECT t.title, p.pub_name
FROM titles t INNER JOIN publishers p
ON t.pub_id = p.pub_id
WHERE t.price > 5 AND t.type = 'business'
ORDER BY t.title
```

خطة التنفيذ:



- في SQL Server يتم قراءة خطة التنفيذ من اليمين إلى اليسار
- والبيانات تقرأ من اليمين إلى اليسار، ومن الأعلى إلى الأسفل

شرح العمليات الظاهرة في الخطة:

1. Clustered Index Scan (on titles):

العملية:

Clustered Index Scan (Clustered)

[titles].[PK_titles]

Cost ≈ 18%

لماذا Index Scan وليس Index Seek ؟

- لدينا في الاستعلام الشرط:

WHERE t.price > 5 AND t.type = 'business'

- لا يوجد فهرس على price أو type
- الفهرس الوحيد هو Clustered Index على title_id
- لا يمكن استخدام title_id لتصفية الصفوف

لذلك SQL Server:

- يقرأ كل صفوف جدول titles
- ثم يطبق شرط WHERE صفّاً صفّاً

ملاحظة:

Index Scan != Table Scan

- Table Scan يعني لا يوجد أي فهرس
- Index Scan يعني يوجد فهرس Clustered هنا (لكن لا يمكن البحث به أي لا يطابق الشرط في where).

2. Clustered Index Seek (on publishers):

العملية:

Clustered Index Seek (Clustered)

[publishers].[PK_publishers]

Cost ≈ 18%

لماذا Seek هنا؟

- لدينا شرط الربط:

t.pub_id = p.pub_id

- pub_id هو Primary Key في publishers
- لكل صف من titles:
- يبحث عن ناشر فقط.
- هذا بحث مباشر باستخدام المفتاح، لذلك يستخدم Index Seek

3. Nested Loops (Inner Join):

العملية:

Nested Loops (Inner Join)

Cost ≈ 0%

لماذا Nested Loops؟

- عدد الصفوف الناتجة من titles بعد التصفية صغير
- لكل صف من titles:
- يتم تنفيذ Seek واحد في publishers

آلية العمل:

- خذ صفاً من titles
 - نفذ Seek في publishers
 - كرر العملية
- ومنه Nested Loop هو الأفضل عندما:

- مجموعة خارجية صغيرة
- يوجد فهرس مناسب في الجدول الداخلي

4. Sort:

العملية:

Sort
Cost $\approx$ 63%
ORDER BY t.title

لماذا Sort مكلف جداً؟

لدينا في الاستعلام:

ORDER BY t.title

- لكن لا يوجد فهرس مرتب على titles
 - البيانات الناتجة من ال JOIN غير مرتبة
- لهذا يقوم SQL Server:

- يجمع كل الصفوف
- يفرزها في الذاكرة

لذلك:

- Sort هو أعلى تكلفة في الخطة

5. SELECT (Root Operator):

العملية:

SELECT

Cost $\approx$ 0%

■ فقط إخراج النتائج النهائية.

يظهر لدينا هنا سؤال:

كيف يمكننا إلغاء عملية الترتيب؟ (التي فعلياً لديها أعلى تكلفة وإلغائها سيؤدي إلى تحسين الخطة):
 لإلغاء **عملية الترتيب (Sort)** من خطة التنفيذ يجب أن نفهم القاعدة الأساسية التالية:
 SQL Server ينفذ Sort إلا إذا لم تكن البيانات مرتبة مسبقاً بالشكل المطلوب
 وبما أن الاستعلام يحتوي على:

ORDER BY t.title

فالطريقة الوحيدة لإلغاء ال Sort هي أن تصل البيانات إلى الخطة مرتبة مسبقاً على title

فكرة قد تخطر ببالنا (لكنها غير صحيحة):

قد نفكر كالتالي:

بما أن الاستعلام يحتوي على ORDER BY t.title ، فلماذا لا نقوم ببناء فهرس على العمود title فقط، وبذلك نلغي عملية الترتيب (Sort)؟
 أي نقوم ببناء الفهرس التالي:

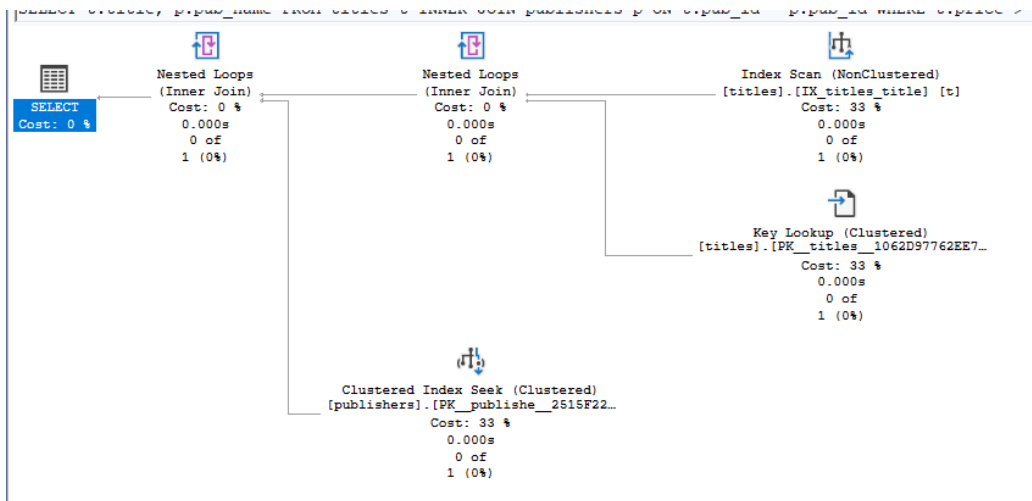
```
CREATE NONCLUSTERED INDEX IX_titles_title
ON titles (title);
```

ظاهرياً يبدو هذا الحل صحيحاً، لأن:

- الفهرس غير المتجمع (Non-Clustered Index) يكون مرتباً حسب العمود title
- وبالتالي نتوقع أن SQL Server سيستخدم هذا الترتيب الجاهز، ولن يحتاج إلى عملية Sort

ماذا يحدث فعلياً في خطة التنفيذ؟

عند تنفيذ الاستعلام بعد إنشاء هذا الفهرس، نلاحظ خطة التنفيذ أصبحت ما يلي:



يتم استخدام الفهرس غير المتجمع على title

- ليس من أجل التصفية (WHERE)
- وإنما فقط للاستفادة من الترتيب

تختفي عملية Sort بالفعل

- لكن هذا ليس نجاحًا حقيقيًا كما سنرى

يظهر بدلاً منها:

- Index Scan على الفهرس غير متجمع
- ثم Key Lookup على الفهرس المتجمع (Clustered Index)

لماذا يحدث Key Lookup؟

الفهرس الذي أنشأناه يحتوي فقط على:

- title
 - والمفتاح الأساسي (title_id) بشكل ضمني
- بينما الاستعلام يحتاج أيضاً إلى:

- price
- type
- pub_id

كما أن شروط التصفية هي:

WHERE price > 5 AND type = 'business'

وهذه الأعمدة غير موجودة في الفهرس.

لذلك يضطر SQL Server إلى:

- قراءة كل صف من الفهرس المرتب
- ثم العودة في كل مرة إلى الفهرس المتجمع لجلب بقية الأعمدة
- وهذا ما يسمى Key Lookup

ومع تكرار هذه العملية لعدد كبير من الصفوف، تصبح التكلفة أعلى بكثير من عملية Sort واحدة

النتيجة: تم إلغاء عملية Sort، لكن الخطة أصبحت أسوأ أداءً.

لماذا هذا الحل غير صحيح من ناحية تصميم الفهارس؟

لأن SQL Server لا يستخدم الفهرس فقط من أجل الترتيب، بل من أجل:

1. التصفية (WHERE)
2. الربط (JOIN)
3. ثم الترتيب (ORDER BY)

فإذا لم يخدم الفهرس شروط WHERE، فإنه سيؤدي إلى:

- عمليات Lookup كثيرة

- زيادة كبيرة في I/O
- وخطة تنفيذ أعقد وأبطأ

الحل الصحيح الأول (الصحيح والأقرب للتفكير):

```
create nonClustered index ix_test on titles(title)
include(price, type, pub_id)
```

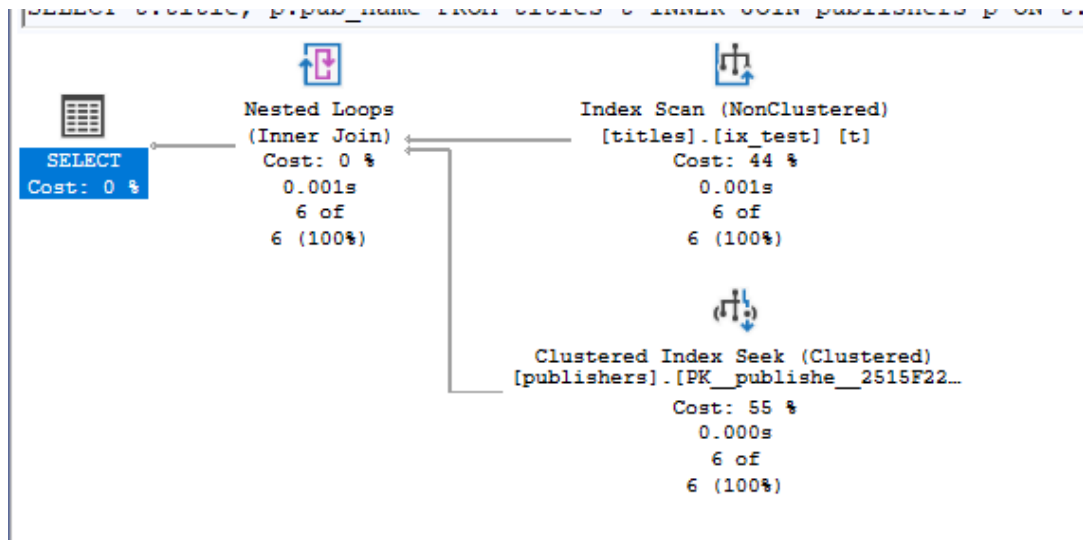
لماذا هذا الحل صحيح؟

1. يحقق شرط ORDER BY مباشرة
لأن الفهرس مرتب على title فلا حاجة لعملية ال sort في الخطة
2. يعتبر فهرس مغطي (Covering Index)

لماذا؟ لأنه غطى كل متطلبات الاستعلام، وبالتالي لا حاجة ل Key Lookup

مغطى؟	جزء الاستعلام
✓	ORDER BY title
✓	SELECT title
✓ (INCLUDE)	WHERE price > 5
✓ (INCLUDE)	WHERE type = 'business'
✓	JOIN pub_id

وتصبح خطة التنفيذ في حال تطبيق هذا الفهرس:

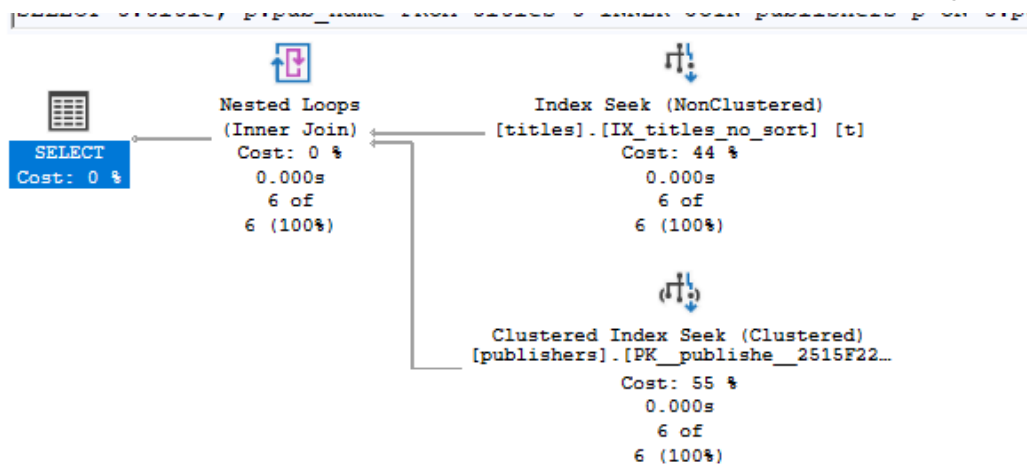


الحل الصحيح الثاني (المحسن):

```
CREATE NONCLUSTERED INDEX IX_titles_no_sort
ON titles (type, title)
INCLUDE (price, pub_id);
```

لماذا هذا الحل أفضل؟

1. يبدأ ب Equality Predicate: type = 'business' ← Index Seek ← يقلل الصفوف مبكراً
 2. يحافظ على الترتيب:
 - tilte يأتي مباشرة بعد type
 - ORDER BY محفوظ
 3. يُستخدم price ك Filter
price > 5
- وخطة التنفيذ لهذا الحل:



The End...

النهاية التي تختارها بشجاعة

أقوى من بداية فُرصت عليك

